Software Project Management (Continued…)

Lecture 10

Prof. H S Behera

Organization of this Lecture

- Overview of Last Lecture
- Staffing
- Scheduling
- Risk Management
- Configuration Management
- Summary

Overview of Last Lecture

- Last lecture we discussed the broad responsibilities of the project manager:
  - Project planning
  - Project Monitoring and Control
- Cost estimation is an important part of project planning.

Overview of Last Lecture

- To estimate software cost:
  - Determine size of the Product.
  - Using size estimation,
    - ✓determine effort needed.
  - From the effort estimation,
    - ✓determine project duration, and cost.

Overview of Last Lecture (CONT.)

- Cost estimation techniques:
  - Empirical Techniques
  - Heuristic Techniques
  - Analytical Techniques
- Empirical techniques are based on systematic guesses made by experts:
  - Expert Judgement
  - Delphi Estimation

Overview of Last Lecture (CONT.)

- Heuristic techniques:
  - assume that different product parameters are related through a simple mathematical expression:
  - COCOMO
- Analytical techniques:
  - derive the estimates starting with some basic assumptions
  - Halstead's Software Science

Overview of Last Lecture (CONT.)

- Staffing level during the life cycle of a software product:
  - follows the Rayleigh curve:
    - ✓Maximum number of engineers required during testing.
    - ✓Number of engineers at any phase depends on the number of parallel activities possible.
- The relationship between schedule change and effort is highly nonlinear.

Overview of Last Lecture (CONT.)

- Team organization:
  - Chief programmer: Suitable for routine development work.
  - Democratic: Suitable for small teams doing R&D type work.
  - Mixed: Suitable for large organizations.

Staffing

- **Staffing in Project Mgmt. involves acquiring, developing, and managing the team required to complete a project efficiently**.
- It entails identifying necessary skills, recruiting, selecting, & organizing staff into roles with defined responsibilities. Effective staffing aligns the right people with project tasks at the right time, enhancing productivity.
- **Staffing is a critical component of the Resource Management plan, ensuring that the right people with the right skills are available at the right time.**

Staffing

- Project Managers usually take responsibility for choosing their team:
  - need to identify and select good software engineers for the success of the project.

Staffing

- A common misconception:
  - one software engineer is as productive as another:
- Experiments reveal:
  - a large variation in productivity between the worst and best in a scale of 1 to 10.
  - Worst engineers even help reduce the overall productivity of the team
    - in effect exhibit negative productivity.

Who is a Good Software Engineer?

- Good programming abilities
- Good knowledge of the project areas (Domain)
- Exposure to Systematic Techniques
- Fundamental Knowledge of Computer Science
- Ability to work in a team
- Intelligence
- Good communication skills:
  - Oral
  - Written
  - Interpersonal
- High Motivation

Who is a Good Software Engineer?

(cont.)

- Studies show:
  - these attributes vary as much as 1:30 for poor and bright candidates.
- Technical knowledge in the area of the project (domain knowledge) is an important factor, determines:
  - productivity of an individual
  - quality of the product he develops.

Who is a Good Software Engineer?

(cont.)

- A programmer having thorough knowledge of database applications (E.g. MIS):
  - may turn out to be a poor data communication engineer.

Scheduling

- **Scheduling in project management** is the process of converting a project plan into an actionable, time-phased operating timetable.
- It defines tasks, dependencies, resources, and deadlines and act as a roadmap to track progress and optimize time.
- **Effective scheduling ensures projects are completed on time and within budget.**

Scheduling contd..

- **Scheduling** is the process of mapping out tasks, timelines, and resources into an operational roadmap to ensure a project is completed on time.
- **While project planning defines, *what* needs to be done, scheduling focuses on *when* it will happen and *who* will do it.**

Scheduling contd..

- **Scheduling** is a crucial phase that breaks down work into manageable activities, defining dependencies, durations, and resources.
- Key techniques like **Gantt charts, PERT, and Critical Path Method (CPM)** are used to visualize the schedule, monitor progress, and meet deadlines.

Key Concepts in Scheduling :

- **Project Scheduling Principles:**

  Involves compartmentalization (breaking tasks down), defining interdependencies (Serial/parallel tasks), time allocation, effort validation, and setting milestones (Key events or key deliverables along a timeline).

- **Steps in Scheduling:**
  - **Work Breakdown Structure (WBS):** Decomposing the project into smaller, manageable activities or tasks.
  - **Define Dependencies:** Identifying which tasks must be completed before others begin (e.g., finishing requirements before designing).
  - **Scheduling Techniques:** Creating visual aids like Gantt Charts (for timelines) and Project Network Diagrams (for dependencies).
  - **Critical Path Method (CPM):** Identifying the longest path of planned activities to the end of the project to determine the minimum project duration.

Common Scheduling Techniques

- **Gantt Charts:** A visual bar chart mapping tasks, durations, and dependencies over time.
- **Critical Path Method (CPM):** Identifies the longest sequence of dependent tasks to determine the minimum project duration.
- **PERT (Program Evaluation and Review Technique):** Estimates project duration by analyzing time required for each task.
- **Fast Tracking/Crashing:** Techniques to shorten the schedule when behind.

Scheduling contd..

- Scheduling is an important activity for the project managers.
- To determine project schedule:
  - Identify tasks needed to complete the project.
  - Determine the dependency among different tasks.
  - Determine the most likely estimates for the duration of the identified tasks.
  - Plan the starting and ending dates for various tasks.

Steps in the Scheduling Process

- **Define Activities:** List all necessary tasks, often derived from the WBS.
- **Sequence Activities:** Identify relationships and dependencies between tasks.
- **Estimate Resources & Durations:** Assign resources and estimate task times.
- **Develop Schedule:** Create the timeline, often using tools like Gantt charts.
- **Monitor & Control:** Track performance against the baseline, updating as needed.

Benefits of Proper Scheduling

- **Organization:** Breaks large, complex projects into manageable pieces.
- **Time Management:** Ensures efficient resource allocation, preventing bottlenecks.
- **Communication:** Keeps team members aligned on deadlines and deliverables.
- **Risk Reduction:** Identifies potential delays early.

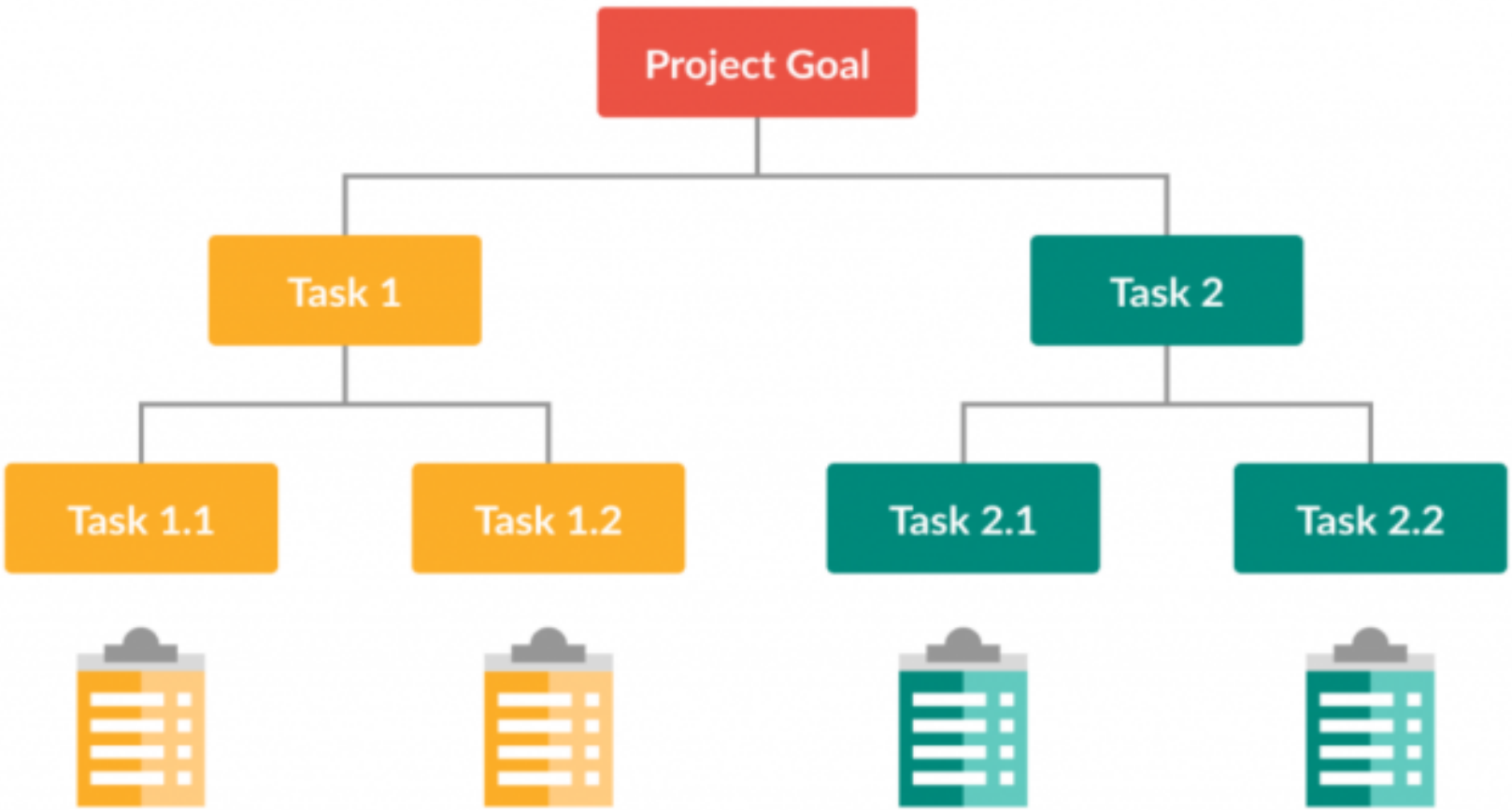
Work Breakdown Structure

- A **Work Breakdown Structure (WBS)** in project management is a hierarchical decomposition of a project into smaller, manageable components, specifically deliverables and work packages. It defines the total project scope, ensures better organization, and aids in scheduling, costing, and resource allocation.

Work Breakdown Structure

- **WBS is a visual, hierarchical tool** used in project management to decompose a project's total scope into smaller, more manageable components. It serves as the "**architectural blueprint**" for a project, defining exactly ***what*** needs to be delivered to meet project objectives.

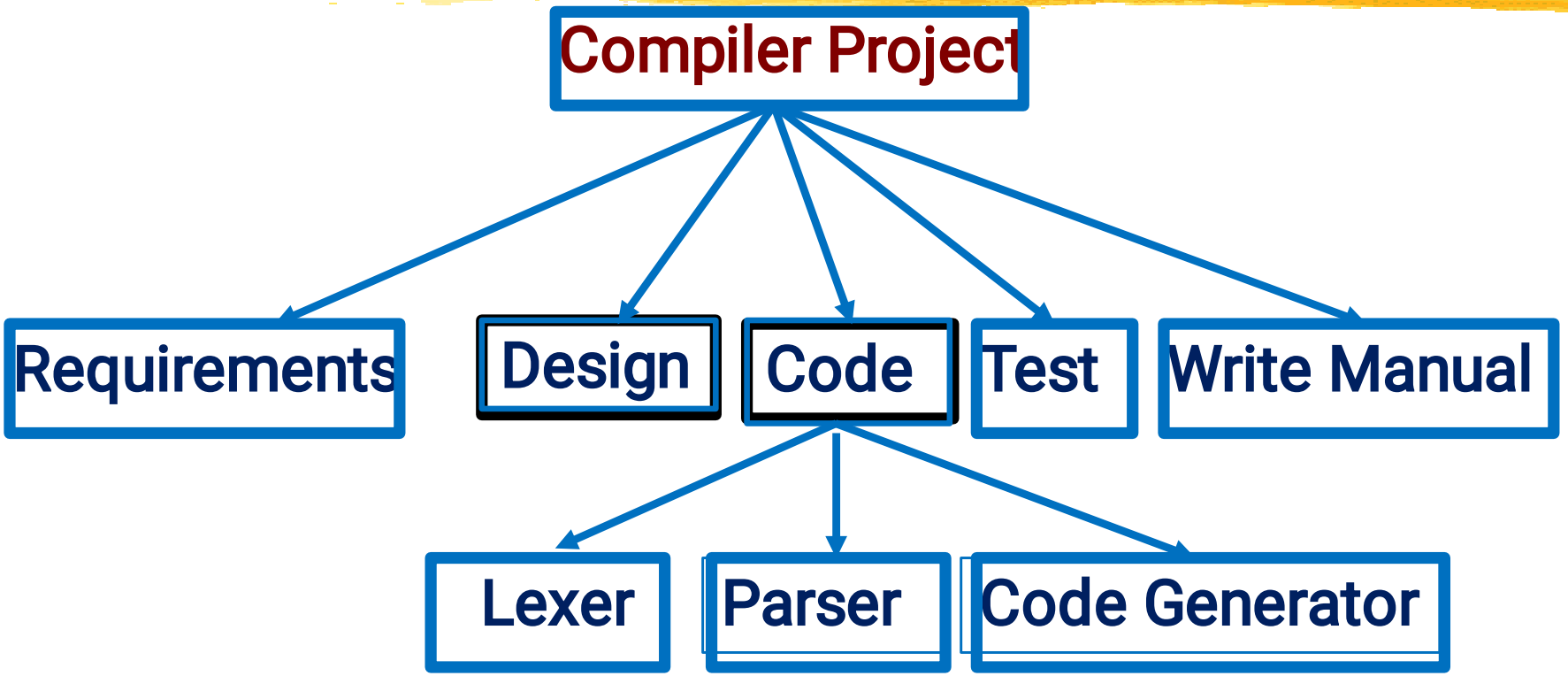
Simple Example to quickly imagine the essence of the Structure:



Work Breakdown Structure

- Work Breakdown Structure (WBS) provides a notation for representing task structure:
  - Activities are represented as nodes of a tree.
  - The root of the tree is labelled by the problem name.
  - Each task is broken down into smaller tasks and represented as children nodes.
- It is not useful to subdivide tasks into units which take less than a week or two to execute.
  - Finer subdivisions mean that a large amount of time must be spent on estimating and chart revision.

Work Breakdown Structure



Activity Networks

- An **activity network** is a graphical tool used in project management **to map the sequence, dependencies, and interrelationships of tasks**.
- It enables calculating the **project's critical path** and overall duration, often using techniques like **PDM** (Precedence diagram method/Activity-on-Node) or **ADM** (Arrow diagram method/Activity-on-Arrow) to optimize scheduling and resource allocation.

Activity Networks contd..

- In project management, an **Activity Network** (or Network Diagram) is a graphical representation of a project's tasks, illustrating their chronological sequence, interdependencies, and the **Critical Path**. It is a foundational tool used to plan, schedule, and monitor progress from initiation to completion.

Activity Networks contd..

- **Activity networks (task networks)** are graphical representations used in software project management to illustrate **task dependencies and flow, forming the basis for scheduling using PERT/CPM**.
- They map out sequential and parallel activities to identify **critical paths, manage slack time**, and determine the minimum project completion time.

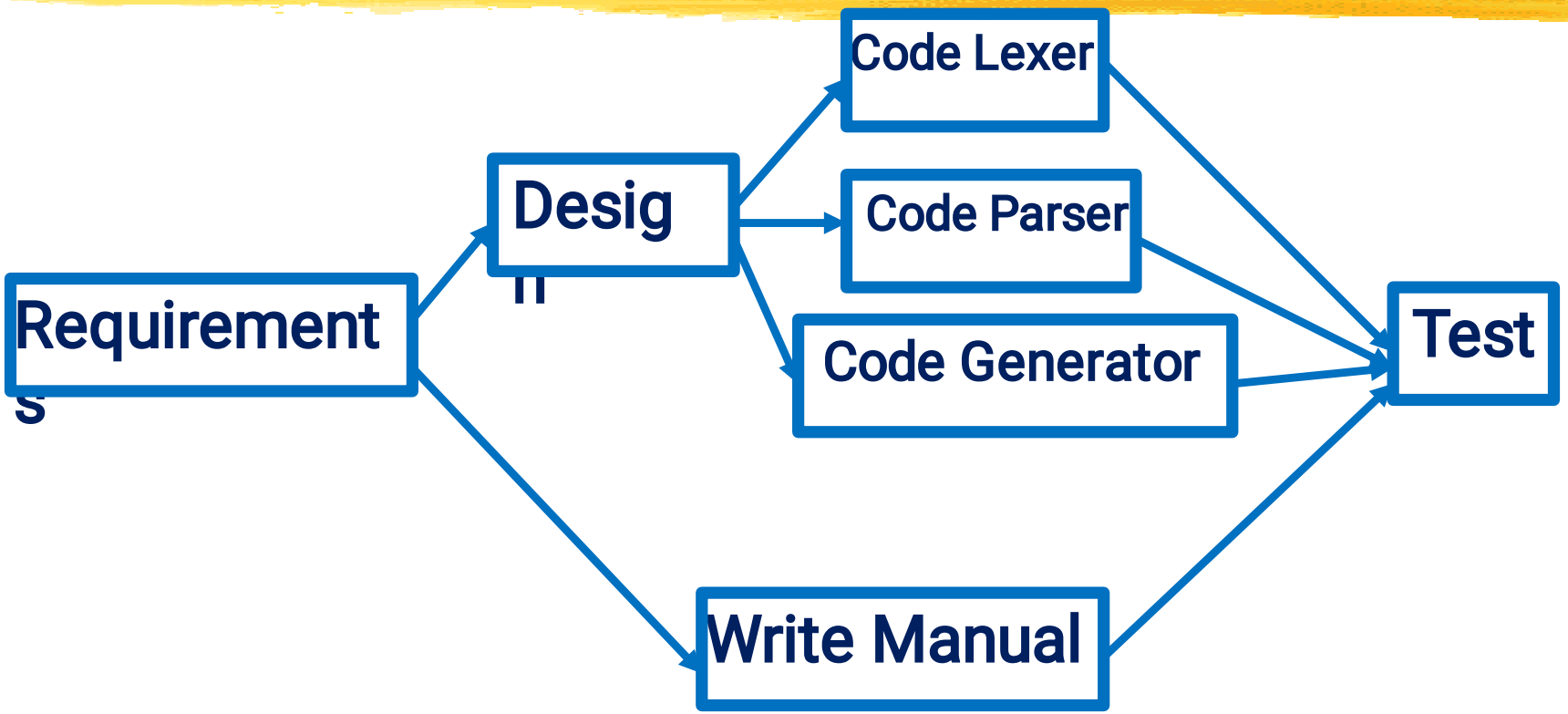
Slack time & Critical Path

- **Slack time** (also known as **float time**) is the total amount of time a task can be delayed without affecting the overall project completion date. **It is calculated as:**
  - **[Latest Start time (LST) - Earliest Start time (EST)] or**
  - **[Latest Finish time (LFT) - Earliest Finish time (EFT)].**
- **Critical Path**: The **Critical path** is the sequence of tasks with **zero slack**. Any delay in these tasks directly delays the entire project.
- **Activities with zero slack are on the critical path, meaning they cannot be delayed without delaying the entire project.**

Activity Networks

- WBS structure can be refined into an activity network representation:
  - Network of boxes and arrows
  - shows different tasks making up a project,
  - represents the ordering among the tasks.
- It is important to realize that developing WBS and activity network
  - requires a thorough understanding of the tasks involved.

Activity Network



Gantt Charts

- A Gantt chart is a visual project management tool that maps project tasks against a timeline, showing start/end dates, dependencies, and progress. It typically appearing as a horizontal bar chart. It displays tasks on the vertical axis and time intervals horizontally, with bars representing task durations, aiding in planning, scheduling, and tracking team workloads.
- It is widely used to schedule and track projects of all sizes by illustrating what needs to be done, when, and by whom.

Gantt Charts

- A **Gantt chart** is a key project scheduling tool that provides a visual timeline using horizontal bars to illustrate task duration, start/end dates, and dependencies.
- It acts as a **tracking tool** that displays progress against planned activities, mapping tasks from a Work Breakdown Structure (WBS) to a chronological schedule

Gantt Charts

- **Gantt chart** is a specialized **bar chart** used to illustrate a project's schedule by visualizing tasks against a timeline. It serves as a **primary tool for project planning, resource allocation, and progress monitoring.**

Gantt Charts

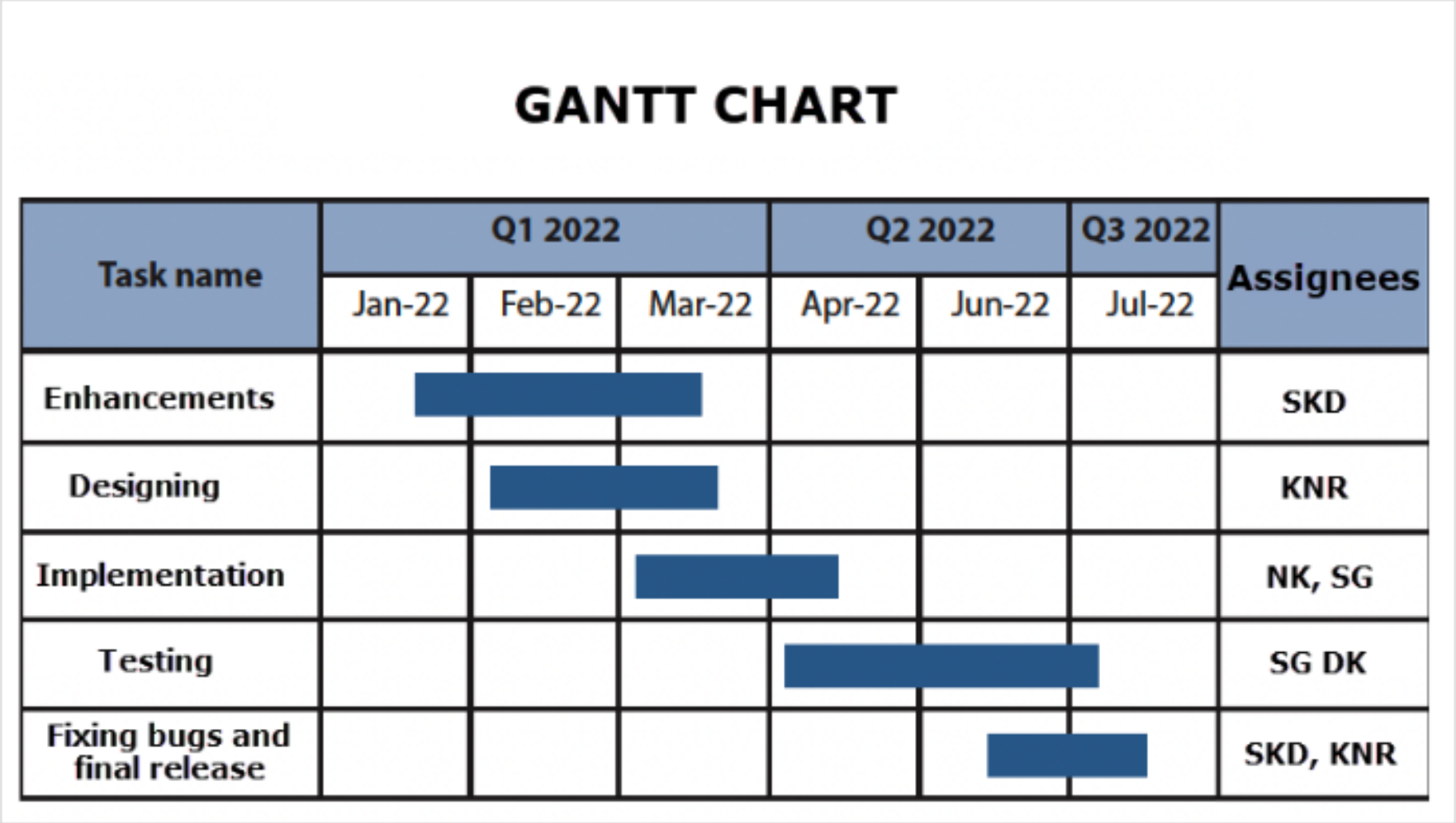
- A **Gantt chart** is a visual project management tool (designed by Henry Gantt, an American mechanical engineer, and social scientist, in the 1910s) that illustrates a project schedule by displaying tasks as horizontal bars across a timeline.
- It is the most widely used chart for scheduling, managing, and monitoring project progress across various industries, from software development to heavy construction.

Gantt Charts

- Named after its developer Henry Gantt.
  - a form of bar chart:
    - ✓each bar represents an activity,
    - ✓bars are drawn against a time line,
    - ✓length of each bar is proportional to the length of time planned for the activity.

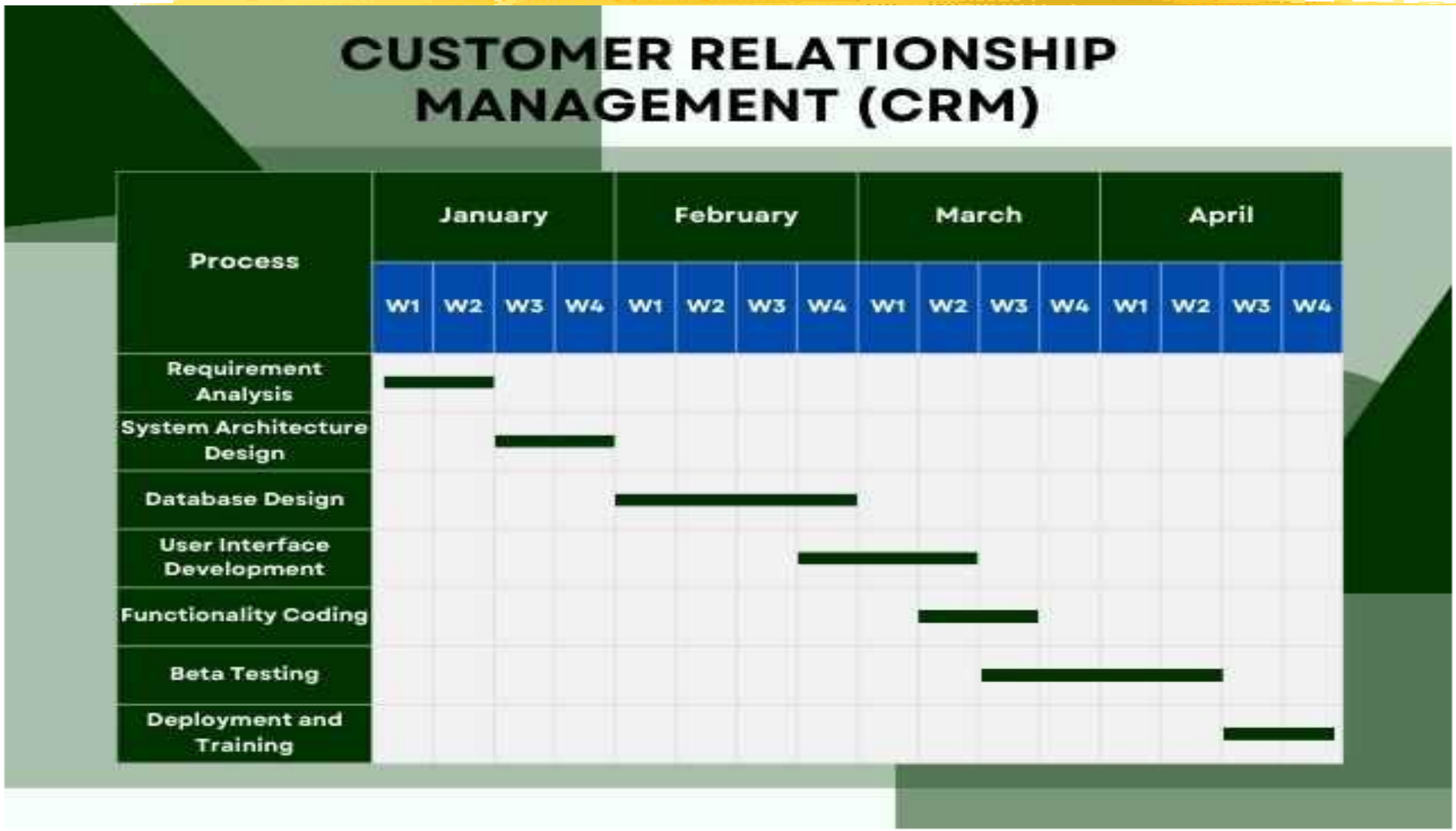
Example of a Gantt Chart for new enhancements in a software:

(quarter wise)



Gantt Chart for CRM

(on the basis of four months and week)

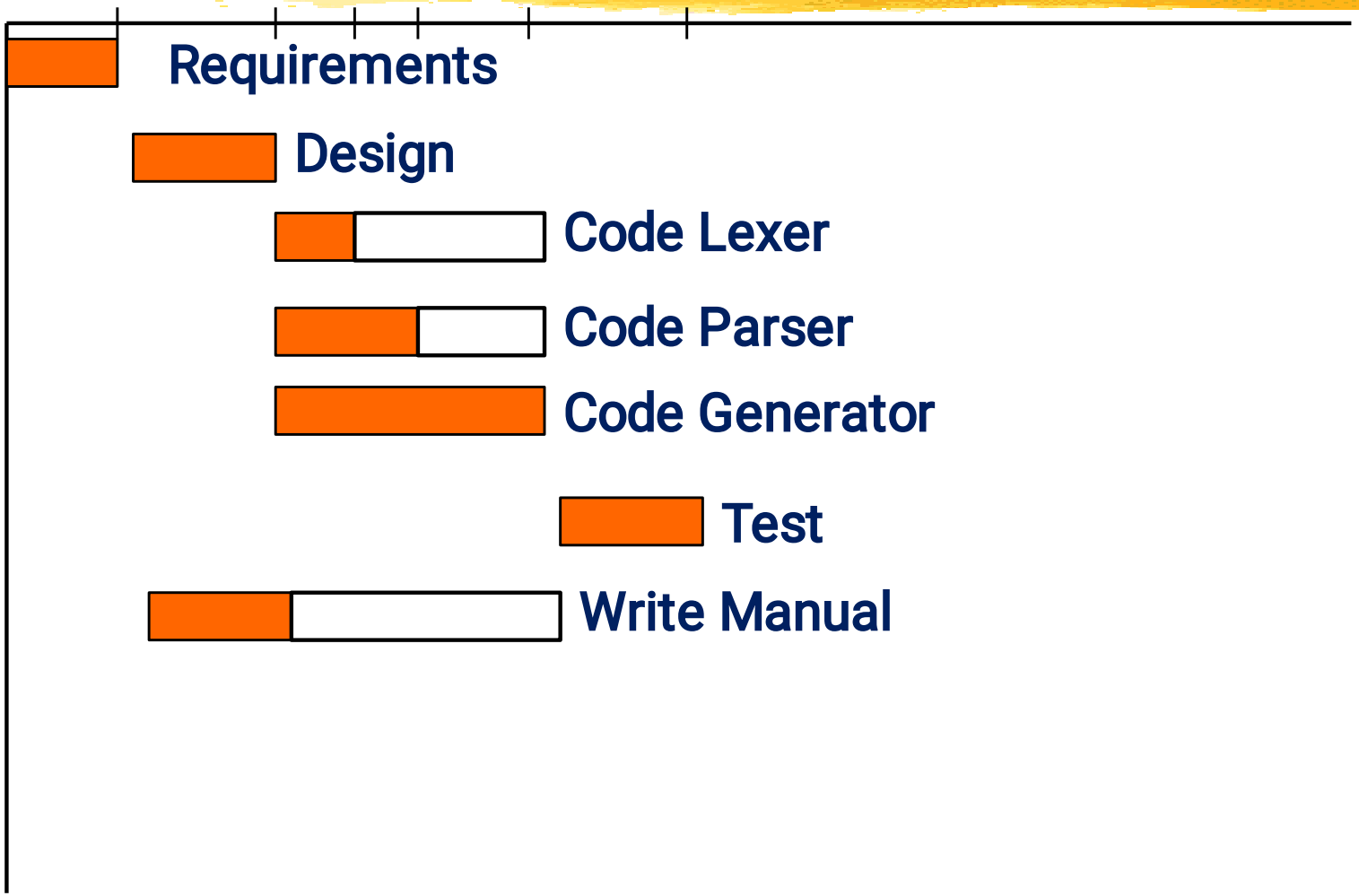


Gantt chart for CRM

Gantt Charts

- Gantt charts are not specific to software engineering.
- Gantt charts used in software project management are:
  - enhanced version of standard Gantt charts.
  - colored part of a bar shows the length of time a task is estimated to take.
  - white part shows the slack time,
    - ✓the latest time by which a task must be finished.

Gantt Chart


Requirements
Design
Code Lexer
Code Parser
Code Generator
Test
Write Manual

Scheduling

- Many managers believe
  - an aggressive schedule motivates the engineers to do a better and faster job.
  - However, careful experiments show:
    - ✓unrealistic aggressive schedules cause engineers to compromise on intangible quality aspects,
      - also cause schedule delays.

Scheduling

- A good way to achieve accuracy:
  - let people set their own schedules.
- Schedule for a large-sized task may take too long:
  - Managers need to break large tasks into smaller ones to find more parallelism
    - can lead to shorter development time.
    - Small-sized tasks help in better tracking

Critical Path

- The **critical path** in project management is the longest sequence of dependent tasks that must be completed on time to meet a project's deadline.
- It represents the shortest possible project duration, with zero slack (or "float") for tasks, meaning any delay in these activities directly delays the entire project.

Critical Path

- **Critical path** is the longest sequence of dependent tasks that must be completed from start to finish to deliver a project on time. It determines the shortest possible duration of the project, as any delay in a task on this path will directly push back the final completion date.

Critical Path

- Task dependencies define a partial ordering among tasks, i.e.
  - Completion of some tasks must precede the starting time of some other tasks.
- A critical path:
  - along which every milestone is critical to meeting the project deadline.
- A <u>Critical Path</u> is a chain of tasks that determine the duration of the project.

Critical Paths

- A critical paths is sequence of tasks such that
  - a delay in any of the tasks will cause a delay to the entire project.
- There can be more than one critical path in a project.
- It is important for the project manager to be aware of the critical paths in a project:
  - can ensure that tasks on these paths are completed on time.

Critical Paths

- Other tasks may have some room for delay without affecting the entire project.
  - If necessary, the manager may switch resources from a noncritical task to a critical task.
- Several software packages are available for automating the scheduling process:
  - Mac Project on Apple Macintosh computer
  - MS-Project on Microsoft Windows Operating System.

PERT Charts

- A **PERT (Program Evaluation and Review Technique) chart** **is a project management tool used to plan, schedule, and coordinate complex tasks by mapping their sequence, dependencies, and duration.**
- **Developed by the U.S. Navy in the 1950s**, it highlights the critical path to determine the shortest possible project completion time.

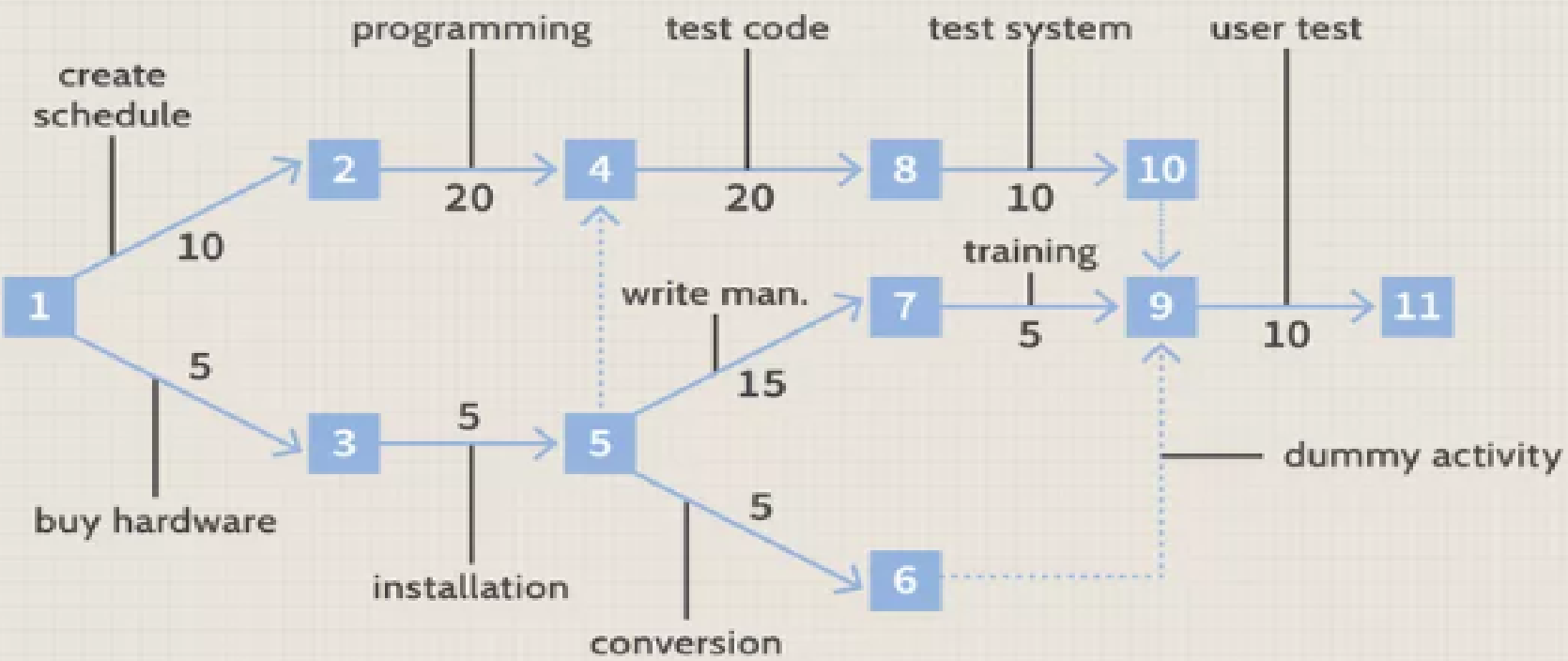
CPM and PERT Charts

- While Gantt charts show the different tasks and their durations clearly:
  - they do not show inter task dependencies explicitly.
  - this shortcoming of Gantt charts is overcome by PERT charts.

PERT Charts

- A PERT chart uses **nodes** (**circles or rectangles**) to represent project **events** or milestones.
- The **nodes are linked by vectors (lines) that represent various tasks**. Dependent tasks are items that must be performed in a specific manner.
- For example, if an arrow is drawn from task 1 to task 2 on a PERT chart, task 1 must be completed before work on task 2 begins.

A well-constructed PERT chart looks like this:



- Numbered rectangles are nodes and represent events or milestones.
- Directional arrows represent dependent tasks that must be completed sequentially.
- Diverging arrow directions (e.g. 1-2 & 1-3) indicate possibly concurrent tasks.
- Dotted lines indicate dependent tasks that do not require resources.

Advantages and Disadvantages of PERT Charts

Pros

- Managers can evaluate time and resources to complete project
- Assets required can be tracked during the course of project
- Incorporates data and information from all departments
- Improves communication
- Useful input for what-if analyses

Advantages and Disadvantages of PERT Charts

Cons

- May include unreliable data or unreasonable estimates
- May not fully communicate project's financial positioning
- Needs a lot of time, labor, and project positioning

Critical Path Management

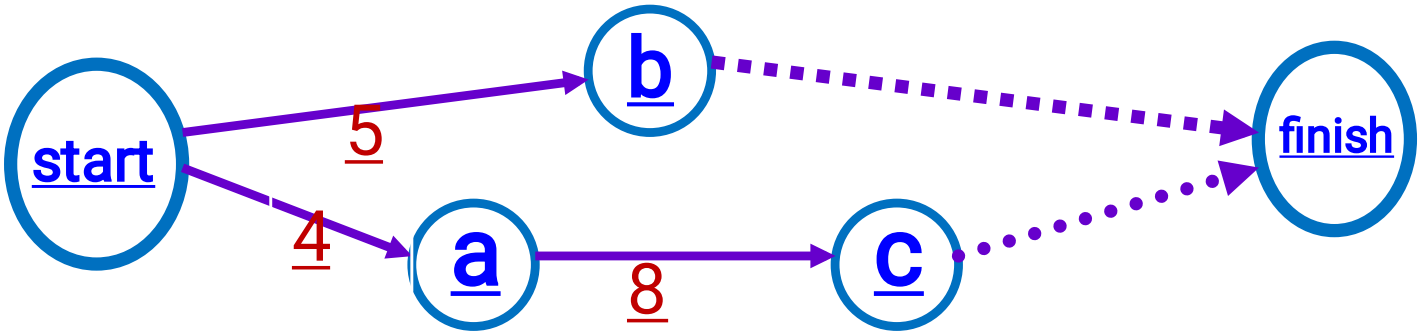
- Critical Path Management(CPM) is a technique for:
  - Identifying critical paths
  - Managing project.
- The CPM technique is not specific to software engineering
  - has a much wider use.

Critical Path Management

- CPM can assist in answering questions like:
  - ✓What are the critical paths in the project?
  - ✓What is the shortest time in which the project can be completed?
  - ✓What is the earliest (or latest) time a task can be started (or finished) without delaying the project?

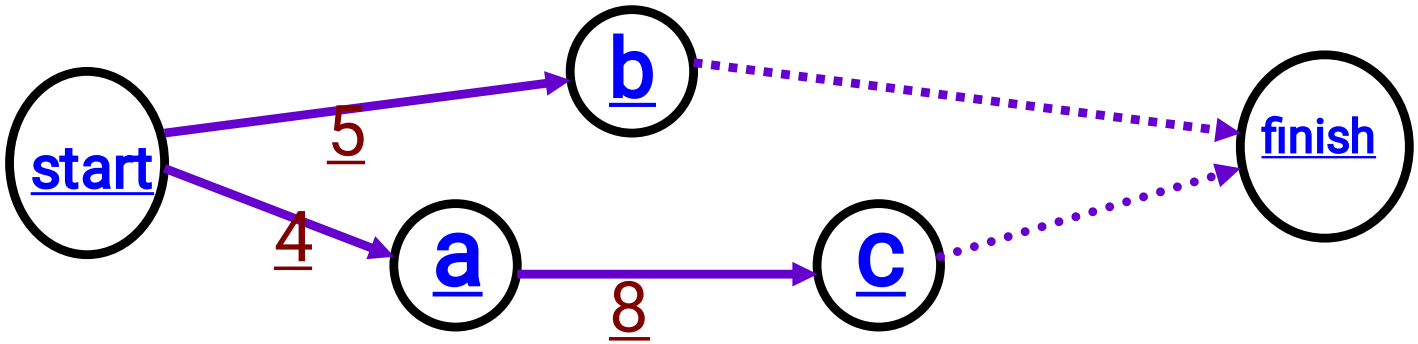
Example

- A project involves three tasks:
  - task a takes 4 hours,
  - task b takes 5 hours
  - task c takes 8 hours.
  - task c cannot commence until task a is completed.
- What is the shortest time in which the project can be completed?



Example

- Clearly, the project continues until task a and then task c complete:
  - which is 12 hours.
  - Task b takes only 5 hours.
  - Task b can have 7 hours of leeway to start and finish.



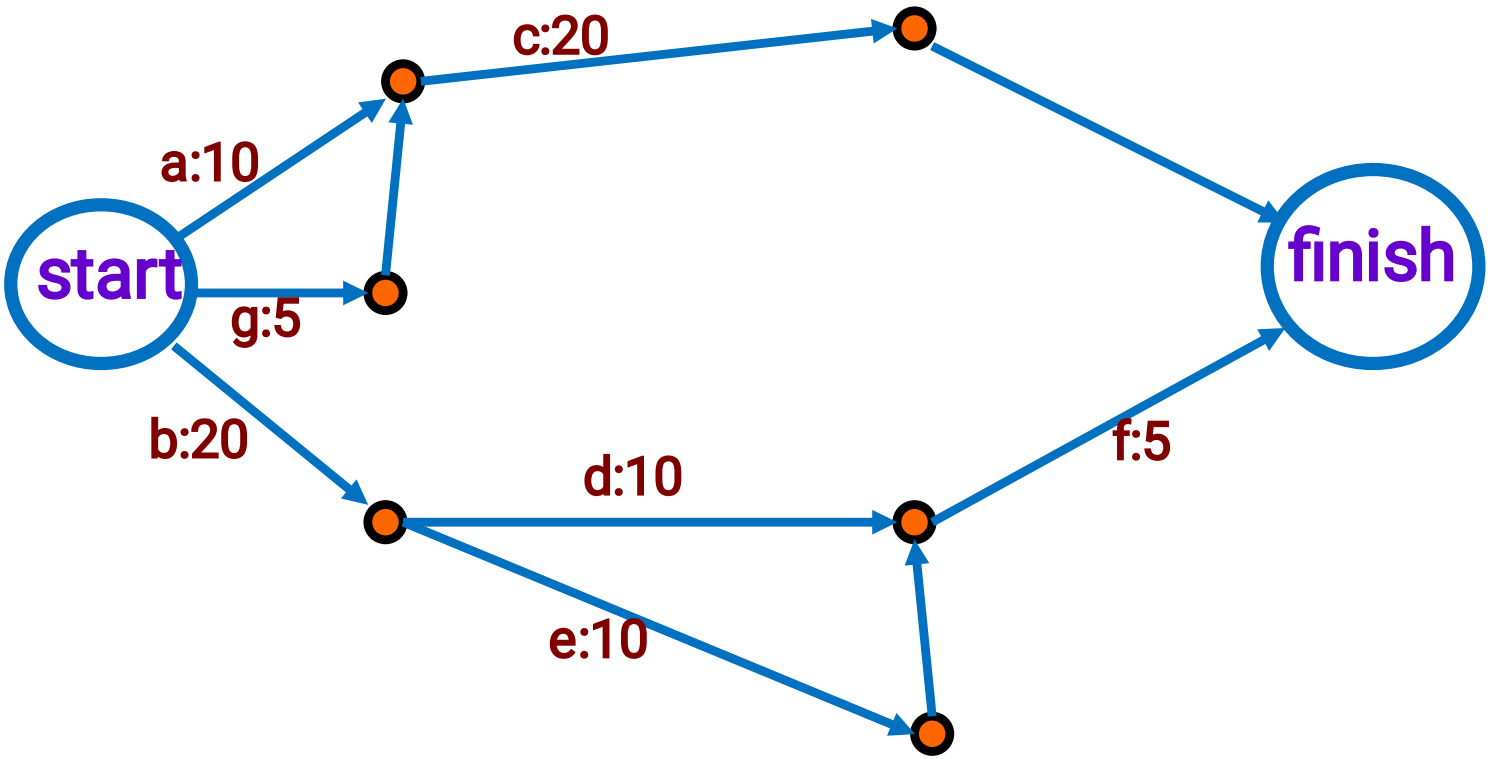
What data do we need to construct a CPM graph?

- To construct a CPM graph,
  - ✓a list of tasks and their durations are required.
  - ✓Also, for each task a list of tasks upon which it depends is required.
  - ✓A task may depend on more than one task.
- Project task details can be given in the form of a table.

Task Table

Task	Duration	Dependents
a	10	
b	20	
c	20	a , g
d	5	b
e	10	b
f	5	h

CPM Graph



How do we work out the various start and finish times for tasks?

- Minimum time to complete project (MT) = Maximum of all paths from start to finish
- Earliest start time (ES) of a task = Maximum of all paths from start to this task
- Earliest finish time (EF) of a task = ES + duration of the task
- Latest finish time (LF) of a task = MT - Maximum of all paths from this task to finish
- Slack time = LS - ES = LF - EF

Start and finish times for tasks.

- Latest start time (LS) of a task = LF - duration of the task

Task	MT	EF	ES	LF	LS
a	10	0	10	15	5
b	20	0	20	25	0
c	20	10	30	35	15
d	5	20	25	30	25
e	10	20	30	30	20
f	5	0	5	15	10

What are the float time (or slack time) of tasks?

- Float time (or slack time) is the total time that a task may be delayed
  - before it will affect the end time of the project.
- The float times indicate the "flexibility" in starting and completion of tasks:
- A critical activity is an activity with zero (0) slack or float time.

What is PERT and how does it work?

- PERT (Program Evaluation and Review Technique) is a variation of CPM:
  - incorporates uncertainty about duration of tasks.
- Gantt charts can be derived automatically from PERT charts.
- Gantt chart representation of schedule is helpful in planning the utilization of resources,
  - while PERT chart is more useful for monitoring the timely progress of activities.

PERT Chart vs. Gantt Chart

- **PERT charts** (Program Evaluation Review Technique) are network diagrams focusing on task dependencies and sequencing, ideal for complex, early-stage planning.
- **Gantt charts** are bar charts illustrating schedules, deadlines, and progress over time, best for tracking execution.
- PERT emphasizes *how* tasks relate; Gantt emphasizes *when* tasks happen.

PERT Chart vs. Gantt Chart

- The primary difference between a **PERT chart** and a **Gantt chart** lies in their focus: a PERT chart is a flow-based network diagram used for **initial planning** and mapping complex task dependencies, while a **Gantt chart** is a timeline-based bar chart used for **tracking progress** and managing schedules during project execution.

PERT Chart vs. Gantt Chart

Key Comparison: PERT vs. Gantt

Feature	PERT Chart (Program Evaluation & Review Technique)	Gantt Chart
Visual Format	**Network diagram** (flowchart) using nodes (tasks) and arrows (dependencies).	**Bar chart** plotted against a horizontal timeline.
Best Use Phase	**Planning phase**: Scoping, identifying dependencies, and estimating duration.	**Execution phase**: Tracking real-time progress, deadlines, and resource allocation.
Main Focus	Visualizing the **logic and sequence** of tasks and the "critical path".	Visualizing the **project schedule**, task durations, and overlapping activities.
Handling Uncertainty	High; uses three-point estimation (optimistic, pessimistic, most likely).	Low; works best with well-defined tasks and stable timelines.
Complexity	Can become cluttered and hard to read for very large projects.	Highly intuitive and easy to read for most stakeholders.

Gantt VS. PERT/CPM

Comparison: Gantt vs. PERT/CPM

Feature	Gantt Chart	PERT/CPM Chart
Primary Focus	Task durations and project timeline	Task interdependencies and critical path
Visualization	Horizontal bar chart	Network of boxes and arrows (activity network)
Best For	Monitoring daily progress and resource loading	Identifying the minimum project completion time

Risk Management

- A risk is any unfavorable event or circumstance:
  - which might hamper successful or timely completion of a project.
- Risk management:
  - concerned with the reduction of the impact of risks.
- Risk management consists of three activities:
  - risk identification,
  - risk assessment, and
  - risk containment.

Risk identification

- To be able to identify various risks:
  - we must categorize risks into different classes.
- Three main categories of risks can affect a software project:
  - project risks
  - technical risks
  - business risks

Project Risks

- Project risks associated with:
  - budget,
  - schedule,
  - personnel,
  - resource, and
  - customer problems.

Technical Risks

- Technical risks concern:
  - requirements specification
    - (e.g ambiguous, incomplete, changing specifications)
  - design problems,
  - implementation problems,
  - interfacing problems,
  - testing, and maintenance problems.
  - technical uncertainty, and technical obsolescence are technical risk factors too.

Business Risks

- Business Risks include:
  - building an excellent product that no one wants,
  - losing budgetary or personnel commitments, etc.
- It is a good idea to have a "company disaster list",
  - a list of all bad things that have happened in the past
  - project managers can jog their mind to see which items their project is vulnerable to.

Risk assessment

- Objective of risk assessment is to prioritize the risks:
  - Likelihood of a risk being real.
  - Consequence of the problems associated with that risk.
- Prioritization helps in handling the most damaging risks first.
  - Priority of a risk is the product of the likelihood of the risk and the consequences of the problems associated with that risk.

Risk Handling

- Three main strategies for risk handling:
  - Avoid the risk: e.g. change the requirements for performance or functionality.
  - Transfer the risk: allocate risks to third party
    - or buy insurance to cover any financial loss should the risk become a reality.
  - Contingency planning: Prepare contingency pans to minimize the impact of the risk.

Risk Handling

- To decide about risk handling options, we must take into account:
  - cost of reducing risk
  - resulting cost saving from risk reduction.

Risk Containment

- Let us see how we can contain an important type of risk:
  - <u>schedule slippage</u>
    - can be dealt with by increasing the visibility of the project.
- Milestones are placed at regular intervals
  - provide a manager with regular indication of progress.

Containing Schedule Slippage

- A milestone is reached,
  - when documentation produced has successfully been reviewed.

Software Configuration Management

- The results (aka deliverables) of any large software development effort consists of a large number of objects:
  - source code,
  - design document,
  - SRS document,
  - test document,
  - project plan (SPMP) document, etc.

Software Configuration Management (CONT.)

- A configuration is a collection of deliverables:
  - being developed for some customer.
- As development proceeds,
  - the components comprising a configuration undergo changes:
  - Even during maintenance, the components comprising a configuration keep changing.

What is configuration management?

- The set of activities through which the configuration items are managed and maintained
  - as the product undergoes its life cycle phases.

Versions

- A configuration for a particular system is called a version:
  - The initial delivery might consist of several versions,
  - more versions might be added later on.
  - For example, one version of a mathematical package might run on a Unix machine,
    - another on Windows NT, and
    - another on Solaris.

Versions

- As a software is released and used by the customers:
  - errors are discovered,
  - enhancements to the functionalities might be needed.
- A new release of the software is an improved system intended to replace an old one.
- Usually a product is described as version m and release n (or as version m.n)

Software Configuration Management

- Existence of variants of a software product causes several problems.
- Suppose you have several versions of the same module, and
  - find a bug in one of them.
  - it has to be fixed in all versions.

Software Configuration Management

- Different objects are accessed and modified by a number of engineers.
- Unless strict discipline is enforced:
  - regarding updation and storage of the objects through some automated tool,
  - several problems appear.

Software Configuration Management

- For example, an engineer might update the module that he has designed ---
  - without informing the engineers who need to interface with this module.
- Or, two engineers may simultaneously carry out changes to different portions of a module:
  - while saving overwrite each other.

Why Configuration Management?

- To be able to identify the exact state of different deliverables at any time.
- To avoid the problems associated with having replicated objects accessible by multiple engineers.
- Controlling concurrent work on a module by different engineers. (Overwriting one engineer's work by another)
- Letting different engineers work on related modules at the same time.
- Keeping track of variants (versions) and helping fix bugs in them.
- Storing versions and revisions efficiently.
- Maintaining revision history (accounting).

Software Configuration Management

- Configuration management helps to:
  - quickly determine the current state of the product
  - change the various components (modifications, revisions, variations etc.) in a controlled manner.
  - maintaining the current up-to-date status of various versions of the software,
  - control and account changes to the product.

Software Configuration Management Activities

- Configuration management is carried out through three principal activities:
  - configuration identification,
  - configuration control,
  - configuration accounting and reporting.

Software Configuration Management Activities

- Configuration identification
  - which parts of the system must be kept track of?
- Configuration control
  - ensures that changes to a component happen smoothly.
- Configuration accounting
  - keeps track of what has been changed, when, and why.

Configuration Identification

- Deliverable objects can be classified into three main categories:
  - controlled,
  - precontrolled,
  - uncontrolled.
- Controlled objects are under configuration control:
  - you must follow some formal procedures to change them.

Configuration Identification

- Precontrolled objects are not yet under configuration control,
  - but may eventually be under configuration control.
- Uncontrolled objects are not and will not be subject to configuration control.

Configuration Identification

- Typical controllable objects include:
  - Design documents
  - Tools used to build the system, such as compilers, linkers, lexical analyzers, parsers, etc.
  - Source code for each module
  - Test cases
  - Problem reports

Configuration Control

- Configuration control is the process of managing changes.
  - It is the part of configuration management that most directly affects day-to-day operations of the developers.
- Once an object goes under configuration control,
  - any further changes requires approval from a change control board (CCB).

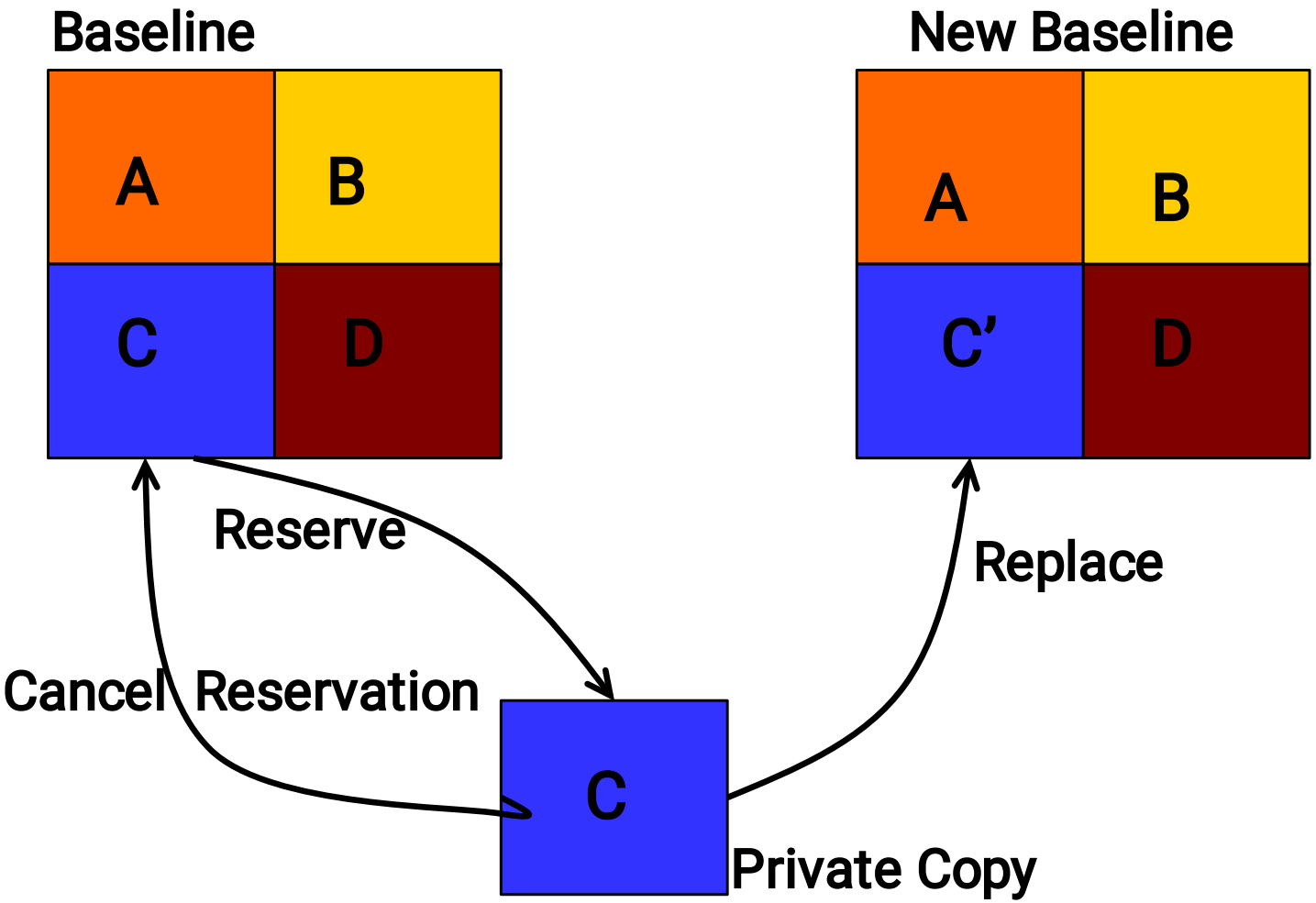
Configuration Control

- The CCB is constituted from among the development team members.
- For every change carried out,
  - CCB certifies several things about the change:
    - Change is well-motivated.
    - Developer has considered and documented the effects of change.
    - Appropriate people have validated the change.

Configuration Control

- An important reason for configuration control:
  - people need a stable environment to develop a software product.
- Suppose you are trying to integrate module A, with the modules B and C,
  - you cannot make progress if developer of module C keeps changing it;
  - this is especially frustrating if a change to module C forces you to recompile A.
- As soon as a document under configuration control is updated,
  - the updated version is frozen and is called a baseline.

Configuration Control



Configuration Control

- This establishes a baseline for others to use.
- Freezing a configuration may involve archiving everything needed to rebuild it.
  - Archiving means copying to a safe place such as a magnetic tape.
- At any given time, a programmer may pay attention to:
  - Current baseline configuration
  - Programmer's private version

SCCS (Source Code Control System)

- SCCS is a configuration management tool:
  - available on Unix systems:
  - helps control and manage text files.
  - also implements an efficient way of storing versions:
    - minimizes the amount of occupied disk space.

SCCS

- Suppose a module is present in 3 versions:
  - MOD1.1, MOD1.2, and MOD1.3.
- SCCS stores the original module MOD1.1
  - together with changes needed to transform it into MOD1.2 and MOD1.3.
  - the modifications are called deltas.

SCCS Features

- Access control facilities provided by SCCS include:
  - facilities for checking components in and out.
  - Individual developers check out components and modify them.
    - after they have changed a module as required and the module has been successfully tested,
    - they check in the changed module into SCCS.

SCCS Features

- Revisions are denoted by numbers in ascending order,
  - e.g, 1.1, 1.2, 1.3 etc.
- It is also possible to create variants of a component
  - by creating a fork in the development history.

Summary

- Project staffing requires careful understanding of the attributes of good software engineers.
- Project scheduling:
  - break down large tasks into smaller logical units,
  - determine dependencies among tasks,
  - determine expected duration of tasks,
  - assign resources, and
  - assign times.

Summary

- Several techniques are available to help in scheduling:
  - Work breakdown structure
  - Activity networks
  - Gantt charts
  - PERT and CPM
- Commercial software tools are available to assist in developing these.

Summary

- It is necessary to determine the critical paths in a project:
  - to meet the timing for critical activities.
- An important project planning activity is risk management:
  - Risk identification
  - Risk assessment
  - Risk containment

Summary

- Configuration management.
  - the set of activities by which a large number of objects are managed and maintained.